

Horizontal Scaling

 systemdesignschool.io/fundamentals/evolution-of-a-web-app

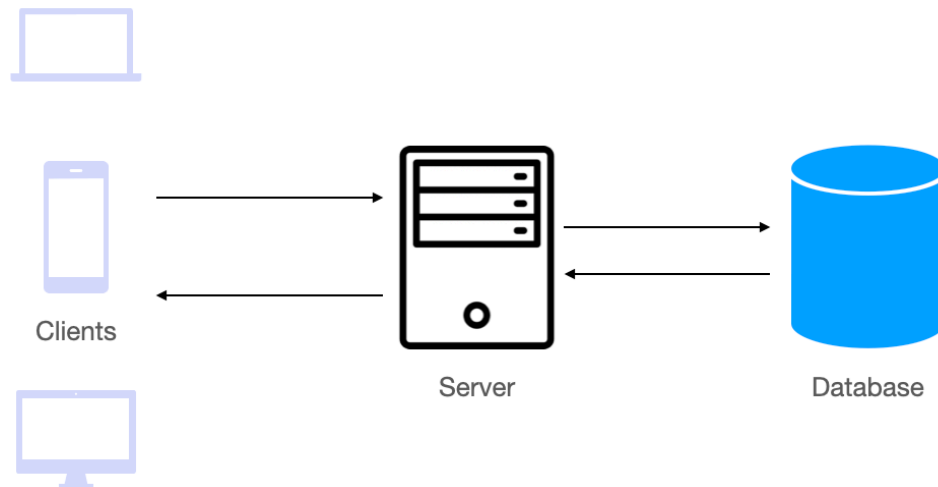


Evolution of a Web App | Single to Scaling

Single-Server Setup

Let's start with everyone's first web app. A plain, simple app that runs on a server written in PHP or Node.js with a database like MySQL or MongoDB behind it. The server takes some requests, asks the database for the information, and returns it to the user's device. There are approximately 100k tutorials out there on how to create such an app.

You have a web server and a database. They probably even live on the same machine. Life is simple and beautiful.

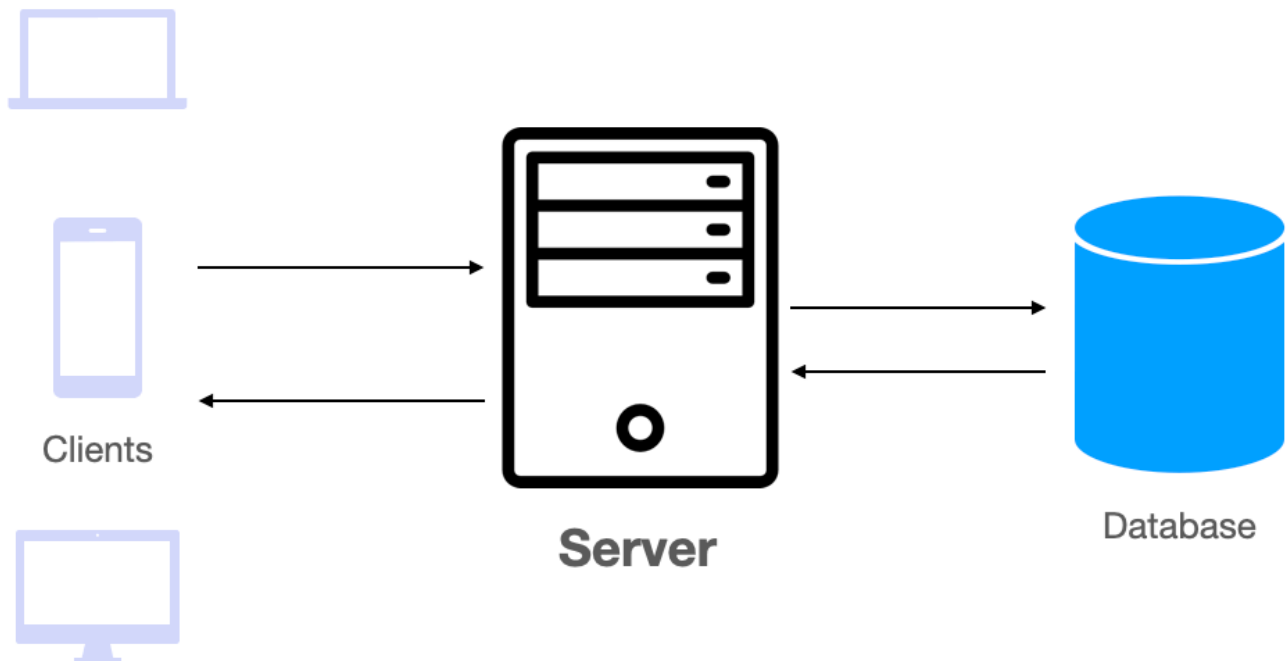


When it comes time to deploy an app like this, you'd rent a VPS from a company like DigitalOcean for five bucks a month, purchase a domain name, set up the DNS to point the domain name to the VPS's IP address, and voila, you just finished your new shiny website for your uncle's online store to sell pet products.

Vertical Scaling

Now your uncle's restaurant is getting really popular, and your \$5/month 1GB memory 25GB SSD DigitalOcean VPS can no longer handle it. What do you do?

The obvious thing to do is to get a larger, beefier server, say \$10/month 4GB 80SSD one. This is called "vertical scaling".



Now let's say your uncle's got great business acumen and his business continues to boom. It has gone from taking orders from friends to shipping within state then to all 50 states and starting to open up international customers. You keep on getting the next larger server. At one point though, this becomes problematic, uneconomical (and slow). Why?

- If that one server goes down, the entire web app goes down. And you are gonna get frantic calls from your angry uncle.
- There is a limit on how big a server can get. Servers get disproportionately expensive as their sizes get large. For example, the largest server AWS offers is the x1.16xlarge instance type (64 vCPUs, 976 GiB of memory).

But let's face it, one server can only go so far. Eventually, you'll hit a wall, and that's where horizontal scaling comes in. Instead of upgrading to a single, super-powerful server, you're going to build an entourage of smaller servers. It's like having a team of personal assistants, all of them dedicated to keeping your website in tip-top shape.

This entourage of smaller servers is put behind a foreman, the "load balancer," whose sole job is to forward requests to other servers to process and then forward the response back to the clients (user devices).

